

Documentacao Tecnica CE Auditavel

1. Como ler este documento

Esta versao foi escrita para ser auditavel.

- Nao usei fontes externas.
- Toda afirmacao abaixo foi extraida do proprio codigo do repositorio.
- Cada bloco termina com referencias para arquivos e linhas especificas.
- Quando ha inferencia minha, eu marco explicitamente como "inferencia".

2. Mapa de fontes

- F1 -> `codigo_julia_exato/supercodigo.jl:1-40`
- F2 -> `codigo_julia_exato/supercodigo.jl:88-264`
- F3 -> `codigo_julia_exato/supercodigo.jl:267-426`
- F4 -> `codigo_julia_exato/supercodigo.jl:1730-1774`
- F5 -> `codigo_julia_exato/exenew2.jl:34-52`
- F6 -> `codigo_julia_exato/exenew2.jl:67-108`
- F7 -> `codigo_julia_exato/exenew2.jl:176-279`
- F8 -> `src/main/java/com/pibic/vrp/controller/VrpController.java:16-55`
- F9 -> `src/main/java/com/pibic/vrp/model/Customer.java:7-35`
- F10 -> `src/main/java/com/pibic/vrp/constraint/SameVehicleConstraint.java:15-115`
- F11 -> `src/main/java/com/pibic/vrp/service/VrpService.java:41-266`
- F12 -> `src/main/java/com/pibic/vrp/service/VrpService.java:120-240`
- F13 -> `src/main/java/com/pibic/vrp/service/VrpService.java:298-369`
- F14 -> `src/main/java/com/pibic/vrp/service/VrpService.java:399-520`
- F15 -> `src/main/java/com/pibic/vrp/service/VrpService.java:553-897`
- F16 -> `src/main/java/com/pibic/vrp/service/VrpService.java:902-1035`
- F17 -> `scripts_ce/dat_to_json_ce.py:44-405`
- F18 -> `scripts_ce/orchestrator_ce.py:44-170`
- F19 -> `scripts_ce/orchestrator_ce.py:361-455`
- F20 -> `scripts_ce/gerar_relatorio_ce.py:1-10`
- F21 -> `scripts_ce/gerar_relatorio_ce.py:114-124`
- F22 -> `scripts_ce/gerar_relatorio_ce.py:346-381`
- F23 -> `scripts_ce/gerar_relatorio_ce.py:421-470`

3. O que esta sendo resolvido

O repositorio implementa um problema de Coleta e Entrega com duas transportadoras, colaboracao horizontal, demandas de entrega e coleta por carrier e custo de deslocamento em matriz. No lado Java, isso aparece no formato `deliveryDemandByCarrier`, `pickupDemandByCarrier` e `allowedCarriers` dentro de `Customer`. No lado Julia, aparece como matrizes `demand`, `coleta` e, em variantes m2m, `demand_tri`. [Fontes: F1, F2, F3, F9, F17]

Inferencia: conceitualmente, o problema e um SCCVRP/VRPSD com dois carriers e possibilidade de compartilhamento de clientes. Essa inferencia vem dos nomes, comentarios e da forma como entrega/coleta simultanea sao modeladas nos dois lados. [Fontes: F8, F10, F11, F17]

4. Como o experimento esta sendo rodado hoje

No pipeline Java, as instancias `.dat` ou `.txt` sao convertidas para JSON por `dat_to_json_ce.py`, depois enviadas para a API por `orchestrator_ce.py`, e os resultados consolidados vao para CSV/Excel. O orquestrador em batch atual executa apenas tres endpoints: `ce-custom`, `ce-custom-no-share` e `ce-c8`. Ele nao chama `ce-c8-no-share`, embora esse endpoint exista no controller. [Fontes: F8, F17, F18, F19]

No pipeline Julia, `supercodigo.jl` le os arquivos `.dat`, extrai os dados, constroi estruturas auxiliares, regrava o proprio arquivo com secoes adicionais e roda varios modelos em sequencia, salvando um `.txt` por modelo. O processamento em lote esta hardcoded para uma pasta de entrada e uma pasta de saida no desktop. [Fontes: F1, F2, F3, F4]

5. Pre-processamento

5.1. Pre-processamento da API Java

O script `dat_to_json_ce.py` parseia:

- `Id`
- `c`
- `n`
- `Q`
- `D`
- `CJ`
- `d`
- `p`
- `cost`

Depois converte isso para o `VrpInput` da API, populando `globalParameters`, `fleets`, `customers` e `costMatrix`. O campo `allowedCarriers` e preenchido a partir de `CJ`. [Fontes: F17]

Ponto auditavel importante: embora `allowedCarriers` exista no modelo `Customer`, o solver Java nao usa esse campo em `VrpService`; o codigo de elegibilidade trabalha sobre `getDeliveryDemandForCarrier` e `getPickupDemandForCarrier`. [Fontes: F9, F11, F12, F13]

5.2. Pre-processamento do Julia

Antes de rodar `run_CE` e `run_CEc8`, o Julia:

- le `custo`, `demand` e `coleta`;
- chama `construir_demand_tri(demand, coleta)`;
- regrava o `.dat` com as secoes `SCCVRPSD` e `SCCVRPSD-M`. [Fontes: F1]

Ponto auditavel importante: `run_CE` e `run_CEc8` usam `demand` e `coleta` diretamente; eles nao usam `demand_tri` na formulacao. Portanto, o preprocessamento aleatorio de `construir_demand_tri` nao interfere diretamente nesses dois modelos. [Fontes: F2, F3, F5, F6, F7]

5.3. Natureza de `construir_demand_tri`

`construir_demand_tri` cria uma matriz tridimensional extra e faz redistribuicao estocastica:

- identifica clientes compartilhados como `hub`;
- divide hubs entre `rotaA` e `rotaB` via `rand(Bool)`;
- escolhe subconjuntos de clientes exclusivos;
- redistribui demanda por proporcoes aleatorias;
- grava o resultado em `demand_tri`. [Fontes: F5, F6, F7]

Conclusao auditavel: isso e preprocessamento para variantes m2m/hub, nao o nucleo da formulacao de `run_CE` e `run_CEc8`. [Fontes: F2, F3, F5, F6, F7]

6. Metaheuristica em Java

6.1. Algoritmo utilizado

A engine de roteamento usada na metaheuristica e `Jsprit`. O codigo nao implementa uma metaheuristica propria do zero; ele configura e chama o algoritmo do `Jsprit` por `Jsprit.Builder.newInstance(problem)`. [Fontes: F11, F14]

Inferencia: do ponto de vista cientifico, o repositorio esta usando a metaheuristica padrao do `Jsprit` em esquema multi-start, e nao um algoritmo inedito implementado manualmente no projeto. [Fontes: F11, F14]

6.2. Como o CE-C8 e modelado na API

`solveWithC8` constroi a matriz de custo, define veiculos com capacidade, cria `Delivery` e `Pickup` separados e adiciona `SameVehicleConstraint`. Para clientes compartilhados em modo `CE`, ele soma as demandas dos dois carriers em um mesmo par de jobs (`d_<cliente>` e `p_<cliente>`). Para clientes exclusivos, adiciona `requiredSkill`; para compartilhados, nao adiciona skill obrigatoria. [Fontes: F11, F12, F10]

Conclusao auditavel:

- no `CE-C8` da API, a visita unica do cliente e garantida por construcao da instancia heuristica;
- a colaboracao aparece como consolidacao da demanda do cliente compartilhado e escolha do carrier executor pelo solver;
- a consistencia entre entrega e coleta e reforcada pela `SameVehicleConstraint`. [Fontes: F10, F11, F12]

6.3. Como o CE-Custom e modelado na API

`solveWithCustomConstraint` identifica clientes compartilhados e gera configuracoes de alocao. As configuracoes incluem:

- todos com carrier 1;
- todos com carrier 2;
- todos separados (`S`);
- alocao por proximidade;
- alocao inversa da proximidade;
- combinacoes extras ou configuracoes aleatorias ate no maximo 8 cenarios. [Fontes: F13, F15]

Depois, para cada configuracao:

- monta um novo problema;
- cria jobs por carrier ou jobs combinados;
- aplica `SameVehicleConstraint`;
- resolve com multi-start;
- guarda a melhor solucao por custo. [Fontes: F13, F15]

Conclusao auditavel: o `CE-Custom` nao reproduz literalmente a formulacao MIP; ele faz uma busca em dois niveis, com enumeracao heuristica de alocoes seguida pelo solver do `Jsprit`. [Fontes: F13, F15]

6.4. Parametros de execucao da metaheuristica

Ha dois esquemas de multi-start:

- `solveWithConstraintMultiStart`: `THREADS = 4`, `VEHICLE_SWITCH = false`, `FAST_REGRET = false`, `maxIterations = 1000`, `seed seed * 1000 + 42`. Esse esquema e usado em `solveWithC8` e nas alocoes de `solveWithCustomConstraint`. [Fontes: F11, F13, F14, F15]
- `solveWithMultiStart`: `THREADS = 8`, `VEHICLE_SWITCH = true`, `FAST_REGRET = false`, `THRESHOLD_ALPHA = 0.1`, `THRESHOLD_INI = 0.05`, `maxIterations = 2000`, `seed seed * 1000 + 42`. Esse esquema e usado no modo single-carrier do `no-share`. [Fontes: F14, F16]

6.5. Pos-processamento da metaheuristica

No `CE-C8`, `mapSolutionC8` deduplica as localizacoes da rota, recalcula `routeCost` via `calculateRouteCost` e devolve esse valor no `RouteDTO`. [Fontes: F14]

No `CE-Custom Multi-Start`, `mapSolutionCustomMultiStart` monta sequencias de atividades e devolve `totalCost`, mas nao preenche `routeCost` por rota. [Fontes: F15]

No `CE-Custom-NoShare`, `mapSolutionSimple` tambem nao preenche `routeCost`. [Fontes: F16]

Consequencia auditavel:

- o `orchestrator_ce.py` calcula `cost_a` e `cost_b` somando `routeCost` no `ce-custom-no-share`;
- como `mapSolutionSimple` nao popula esse campo, os custos por carrier desse cenario podem sair zerados ou incorretos no relatorio. [Fontes: F16, F18, F21, F22]

7. Metodo exato em Julia

7.1. Solver utilizado

O metodo exato usa `JuMP + Gurobi`, com `TimeLimit = 5` e `MIPGap = 0.00` em `run_CE` e `run_CEc8`. [Fontes: F2, F3]

Conclusao auditavel: se o status voltar `TIME_LIMIT`, a comparacao correta nao e "otimo exato vs heuristica", mas sim "melhor incumbente do Gurobi com gap reportado vs heuristica". O proprio codigo grava `objective_value`, `objective_bound` e `relative_gap`. [Fontes: F2, F3]

7.2. Estruturas e variaveis

Nos dois modelos principais, aparecem:

- `setN` para clientes;
- `setC` para carriers `A` e `B`;
- `nos = [setN; setC]`;
- `x[nos, nos, setC]` binaria;
- `z[setN, setC, setC]` binaria;
- `l[nos, nos, setC, setN]` continua;
- `m[nos, nos, setC, setN]` continua. [Fontes: F2, F3]

Inferencia: `x` modela arcos da rota, `z` modela atribuicao de demanda de origem para carrier executor, e `l/m` modelam fluxos de entrega e coleta para controle de capacidade. Essa inferencia e sustentada pela forma das restricoes `c2`, `c4`, `c5`, `c6`, `c7mod` e pelos comentarios do proprio projeto Java. [Fontes: F2, F3, F10]

7.3. Restricoes principais em `run_CE`

Em `run_CE` aparecem explicitamente:

- `c2`: atribuicao unica via `z`;
- `c3`: conservacao de fluxo de visita;
- `c4`: ligacao entre visita e atribuicao;
- `c7mod`: capacidade;
- `c8`: visita unica global;
- `c6`, `c6linha`, `c62`, `c62linha`: conservacao de fluxo de `l` e `m`;
- `c5`, `c5linha`: injecao/fechamento dos fluxos no deposito;
- objetivo de minimizacao do custo total em `x`. [Fontes: F2]

7.4. O que `run_CEc8` realmente faz

`run_CEc8` repete quase toda a estrutura de `run_CE`, com as mesmas variaveis, mesmas familias de restricoes de fluxo, mesma capacidade e mesmo objetivo. Porem, o codigo dessa funcao nao contem a restricao `c8`. [Fontes: F2, F3]

Conclusao auditavel e central:

- `run_CE` e o modelo com `c8`;
- `run_CEc8` esta sem `c8`.

Logo, pelo codigo atual, a nomenclatura esta invertida ou, no minimo, ambigua. [Fontes: F2, F3]

8. Comparacao auditavel entre `run_CE` e `run_CEc8`

8.1. O que e igual

As duas funcoes compartilham:

- `JuMP + Gurobi`;
- `TimeLimit = 5`;
- `MIPGap = 0.00`;
- conjuntos `setN`, `setC`, `nos`;
- variaveis `x`, `z`, `l`, `m`;
- objetivo de custo;

- restricoes `c2`, `c3`, `c4`, `c5`, `c5linha`, `c6`, `c6linha`, `c62`, `c62linha`, `c7mod`;
- escrita de status, objetivo, bound, gap e variaveis positivas em arquivo texto. [Fontes: F2, F3]

8.2. O que muda

A diferenca relevante e so uma, mas ela e estrutural:

- `run_CE` tem `c8` em `supercodigo.jl:171`;
- `run_CEc8` nao tem nenhum bloco correspondente de `c8`. [Fontes: F2, F3]

8.3. Leitura cientifica correta

Se a sua comparacao experimental for "com visita unica" vs "sem visita unica", entao:

- `run_CE` e o cenario com visita unica;
- `run_CEc8` e o cenario relaxado sem essa restricao explicita. [Fontes: F2, F3]

Se a sua comparacao experimental for "metaheuristica vs exato com C8", o pareamento metodologicamente mais limpo e:

- Java `solveWithC8` / endpoint `/ce-c8`;
- Julia `run_CE`. [Fontes: F8, F11, F12, F2]

9. Comparacao auditavel entre metaheuristica e metodo exato

9.1. O que esta proximo entre os dois

Os dois lados tentam representar:

- capacidade do veiculo;
- entrega e coleta simultaneas;
- colaboracao entre carriers;
- decisao de qual carrier executa o atendimento;
- cenario com visita unica no modo `CE-C8` / `run_CE`. [Fontes: F2, F10, F11, F12, F13]

9.2. O que nao e equivalente um-para-um

No Julia, a colaboracao esta modelada explicitamente pela variavel `z[i,r,s]`. No Java, ela aparece por construcao operacional:

- agregacao de jobs compartilhados em `solveWithC8`;
- enumeracao de alocoes em `solveWithCustomConstraint`. [Fontes: F2, F3, F10, F12, F13, F15]

No Julia, `c8` e uma restricao matematica explicita. No Java `CE-C8`, a visita unica e obtida por construcao do conjunto de jobs e pela amarração de `Delivery/Pickup` no mesmo veiculo. [Fontes: F2, F10, F12]

No Java `CE-Custom`, a propria implementacao admite o caso `S` de clientes compartilhados separados e ainda reporta quantas localizacoes ficaram com visita unica ou multipla. Isso mostra que esse modo nao impõe visita unica global por definicao. [Fontes: F15]

9.3. Possivel diferenca estrutural de numero de rotas

No Julia, a formulacao principal indexa `x` por carrier, nao por veiculo individual. No Java, os problemas sao construidos com `FleetSize.INFINITE`. [Fontes: F2, F3, F11, F15, F16]

Inferencia: isso sugere que a estrutura de roteamento admitida pelo Java pode nao coincidir exatamente com a leitura mais estrita do MIP em termos de numero de rotas/veiculos. Como o codigo nao documenta formalmente essa equivalencia, essa diferenca deve ser declarada quando voce comparar cenarios. [Fontes: F2, F3, F11, F15, F16]

10. Pontos criticos para a sua comparacao experimental

10.1. Ponto critico 1

`run_CEc8` nao e o modelo "com C8"; pelo codigo atual, o modelo com `c8` e `run_CE`. [Fontes: F2, F3]

10.2. Ponto critico 2

O batch principal da metaheuristica nao roda `ce-c8-no-share`, embora a API ofereca esse endpoint e o gerador de relatorio tenha abas para ele. [Fontes: F8, F18, F19, F20, F23]

10.3. Ponto critico 3

`allowedCarriers` / `CJ` entra na conversao para JSON, mas nao entra de fato no solver Java. [Fontes: F9, F11, F12, F13, F17]

10.4. Ponto critico 4

Os custos por carrier em `ce-custom-no-share` podem estar errados no relatorio, porque o relatorio soma `routeCost`, mas `mapSolutionSimple` nao preenche `routeCost`. [Fontes: F16, F18, F21, F22]

11. Recomendacao final, com lastro em codigo

Se voce quer comparar "metaheuristico com visita unica" vs "metodo exato com visita unica", use:

- `/ce-c8` no Java;
- `run_CE` no Julia. [Fontes: F8, F11, F12, F2]

Se voce quer comparar "cenario relaxado sem visita unica estrita", use:

- `/ce-custom` no Java;
- `run_CEc8` no Julia.

Mas nessa segunda comparacao, documente explicitamente que as formulacoes nao sao identicas:

- no Java ha enumeracao heuristica de alocoes;
- no Julia ha um unico modelo MIP relaxado. [Fontes: F13, F15, F3]

12. Resumo em uma frase

Auditando o codigo exato e a API atual, a comparacao mais correta para "metaheuristica vs exato" e `CE-C8` (Java) contra `run_CE` (Julia), porque `run_CEc8` esta sem a restricao `c8` apesar do nome. [Fontes: F2, F3, F8, F11, F12]